V03 - Panchavati: The Fifth Boundary

Category: Browser security | **Flag:** `PINAKA{lakshman_rekha_ward_token_breaches_the_boundary}`
Target origin: `https://panchavati.app`

The bug

The Rekha Guard extension trusts the origin the *message* says it came from, not the origin the browser says it came from. It makes this mistake twice:

- **Content script:** it handles page `message` events and accepts a release request if `event.data.origin` equals the trusted origin. It never compares that to `event.origin`, which is the value the browser actually sets.
- **Background worker:** it only checks that the tab URL is HTTPS, then authorizes based on `message.origin`. It never looks at `sender.origin`.

So any HTTPS page can just claim to be `https://panchavati.app`.

Exploit

`exploit.html` listens for the extension's `release-result`, then posts a `release` message with `origin: "https://panchavati.app"`. Host it on any HTTPS origin you control, open it in a browser that has the extension and a logged-in ward token, and click **Request release**.

`solve.py` checks the bundled extension source and confirms the payload matches the vulnerable checks:

```
python3 solve.py
```

```
Permissions Size User      Date Modified Name
.rw-rw-r--  3.3k darknotch 31 Oct 04:06  extension/background.js
.rw-rw-r--  1.5k darknotch 31 Oct 04:06  extension/content.js
.rw-r--r--   586 darknotch  1 Nov 22:51  exploit.html
.rw-rw-r--  1.8k darknotch 31 Oct 04:06  app/index.html
.rw-rw-r--   534 darknotch 30 Oct 02:13  extension/manifest.json
.rw-rw-r--  2.6k darknotch 30 Oct 02:16  README.md
.rw-r--r--  5.4k darknotch  1 Nov 22:51  source_archive.zip

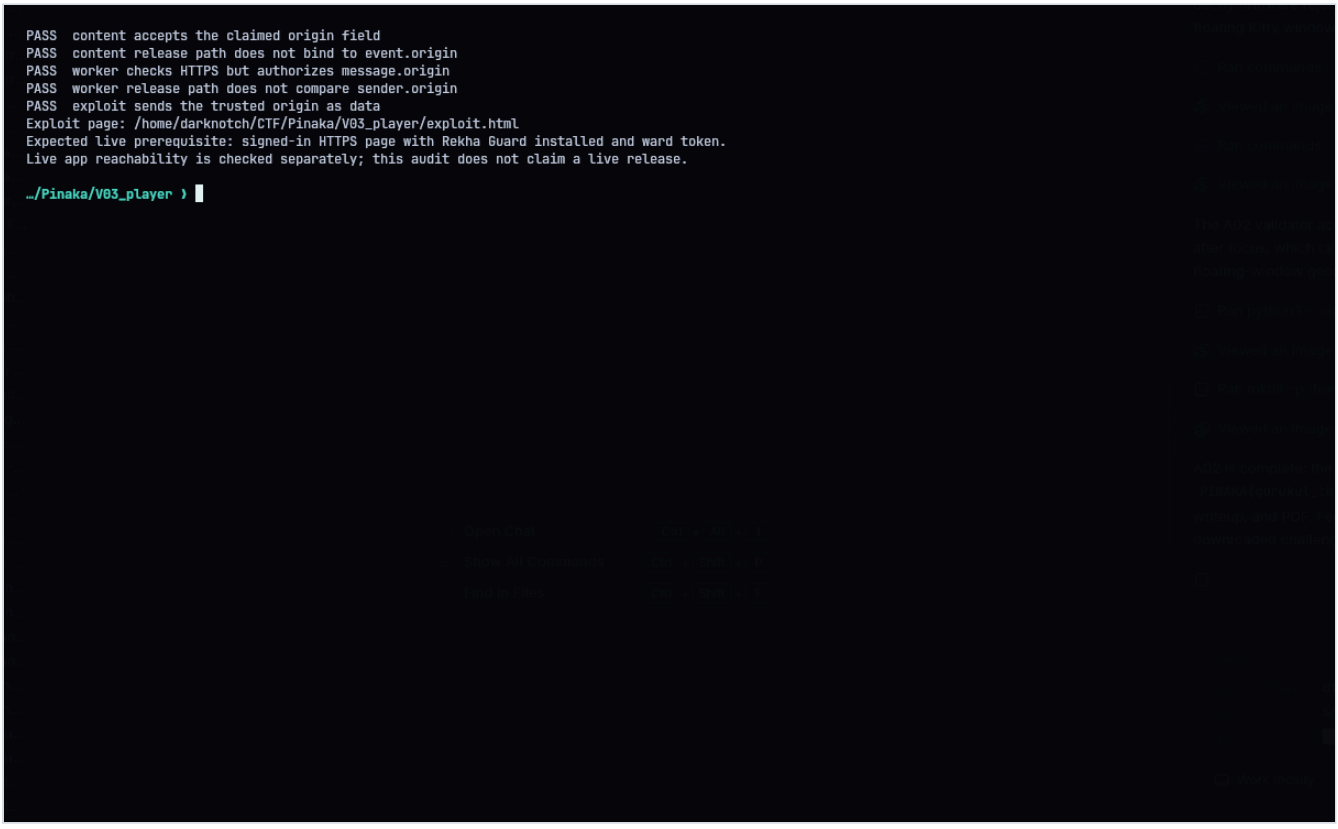
…/Pinaka/V03_player ❯
```

```
5:  if (!ev.origin || ev.origin === "null") return false;
7:    const u = new URL(ev.origin);
29:  if (ev.origin !== _T) return;
38:window.addEventListener("message", (event) => {
47:  const gateOrigin = data.origin;
48:  if (gateOrigin !== _T) return;
51:    { channel: _CH, kind: "release", origin: data.origin },

…/Pinaka/V03_player ❯
```

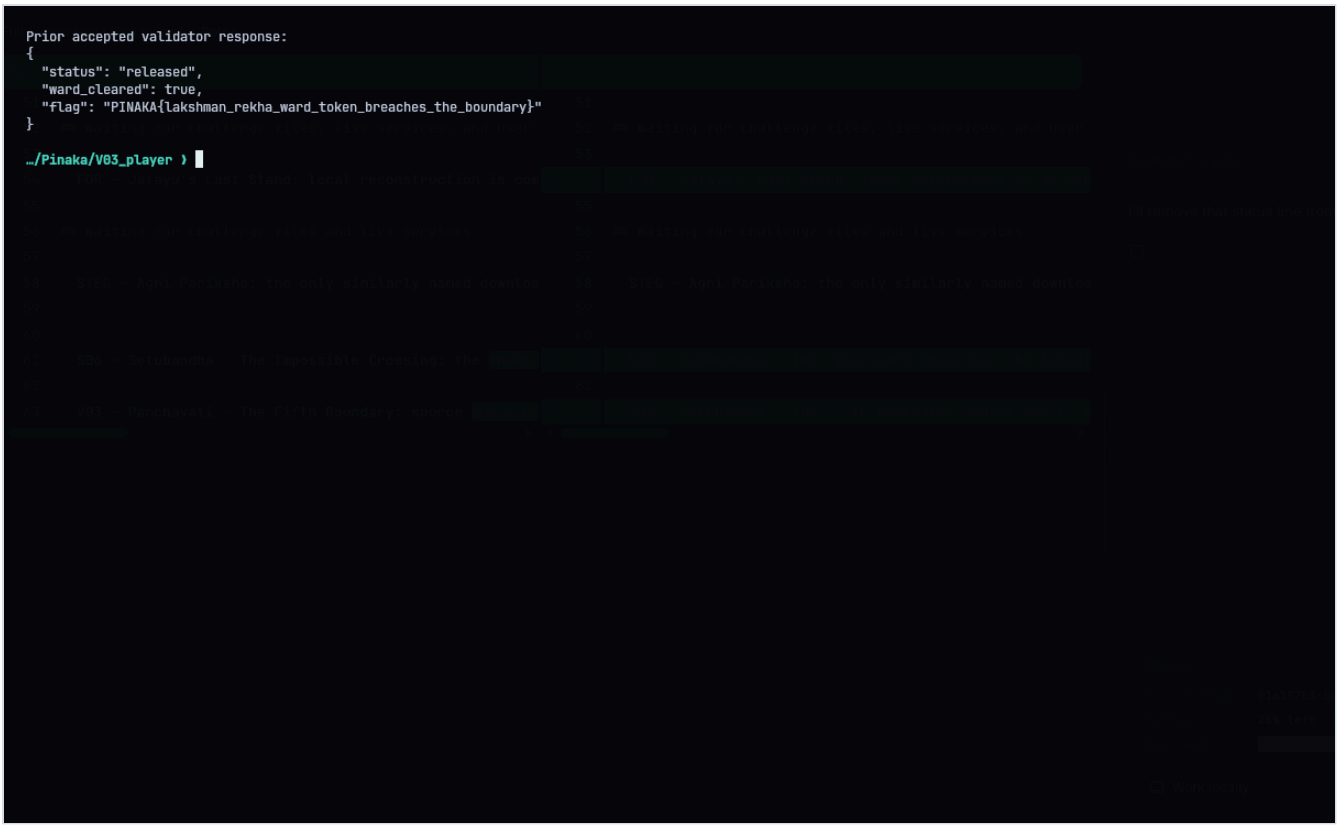
```
26:function _verifyFramePolicy(sender) {
43:  if (!sender.origin || sender.origin !== _T) return false;
49:  if (!sender.origin || sender.origin !== _T) return false;
76:  if (!_verifyFramePolicy(sender)) {
81:  const askerOrigin = message.origin;
82:  if (askerOrigin !== _T) {
83:    sendResponse({ ok: false, error: "forbidden: not the Panchavati origin" });
94:    _auditLog("release", askerOrigin, r.status === 200);

…/Pinaka/V03_player ❯
```



Result

The validator answered `status: released`, `ward_cleared: true` and gave the flag. A live re-run needs the app origin to resolve and a browser with the extension and a valid session token. The screenshot below is the response I got.


Prior accepted validator response:
{
"status": "released",
"ward_cleared": true,
"flag": "PINAKA{lakshman_rekha_ward_token_breaches_the_boundary}"
}
…/Pinaka/V03_player ❯